

PESTO: **P**arametric **E**xecution-path- **S**ensitivity for **T**est **O**ptimization

CS453 Project Presentation

Background

- Negative tests

Tests that validate how a SUT handles invalid input, ensuring it fails gracefully (e.g. via exceptions) without crashing

- Meta-programs

Programs that take other programs as inputs, e.g. compilers or interpreters.

Background

- Negative tests **for meta-programs**

Programs that validate how **a meta-program** handles invalid **programs**, ensuring it **raises an expected exception**.

- Example: CPython negative test = python program string

```
self.assertRaises(TypeError, eval, "1 + 'x'")  
self.assertRaises(TypeError, eval, "sum('z', 3)")
```

Motivation

A well-designed set of negative tests lets you:

- Ensure existing error-handling behavior is preserved (regression testing)
- Ensure other implementations (e.g. PyPy) agree
- Preferably, achieve this with a small #. of tests and short runtime.

Problem Statement

Generate an optimal negative test suite, that
1) sufficiently cover diverse invalid conditions,
with a 2) small number of 3) minimal programs.

Core Idea: Trace Sensitivity

- Each negative test has **exactly one reason** it is invalid
 - Delta-debugging can reduce invalid programs to their invalid reasons
 - However, exceptions do not map 1-to-1 to invalid reasons
 - Example:
 - `1 + 'x'`
 - `2 + 'y'`
 - `sum('z', 3)`
- We need additional information

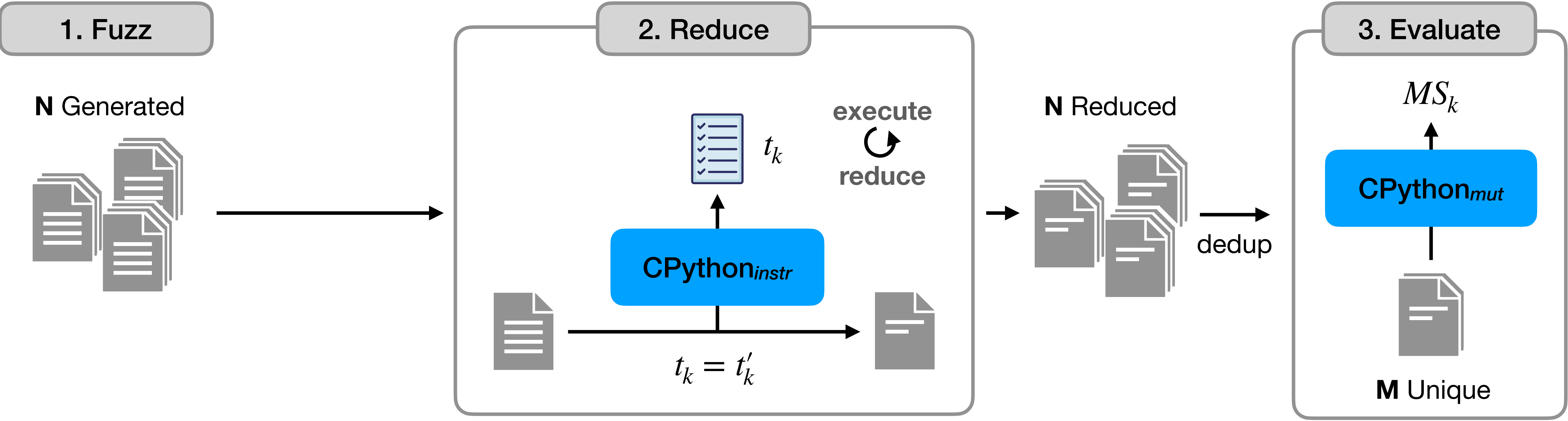
`TypeError: unsupported operand
type(s) for +: 'int' and 'str'`

PESTO

We present **PESTO** (**Parametric Execution-path Sensitivity for Test Optimization**),
a framework that produces an optimal negative test suite
using **trace-sensitivity** to preserve **diversity**
while reducing well-formed programs **minimally**

Methodology

PESTO Overview



Methodology

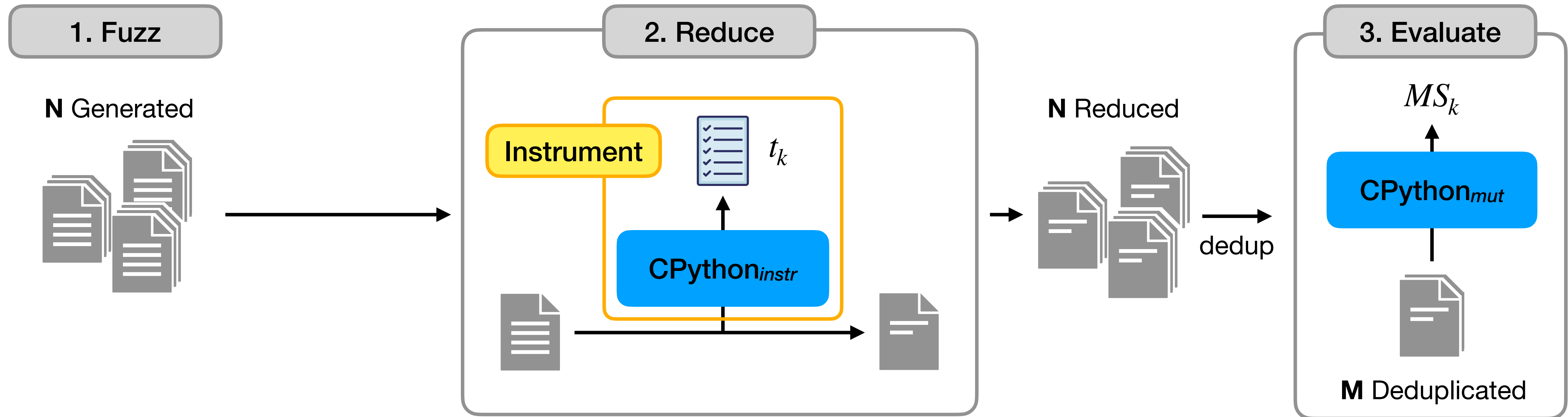
Fuzzing

- Purely random fuzzing produces mostly syntax errors, so constrained fuzzing is needed to generate diverse negative tests
- Grammarinator ([1] ISSTA 2018)
 - Automatically generates programs based on a pre-defined grammar
 - Requires only the grammar definition as input
- Discard programs with syntax errors and programs with no errors

Methodology

Instrumentation

$t_k := \text{exception} + \text{last } k \text{ call traces}$

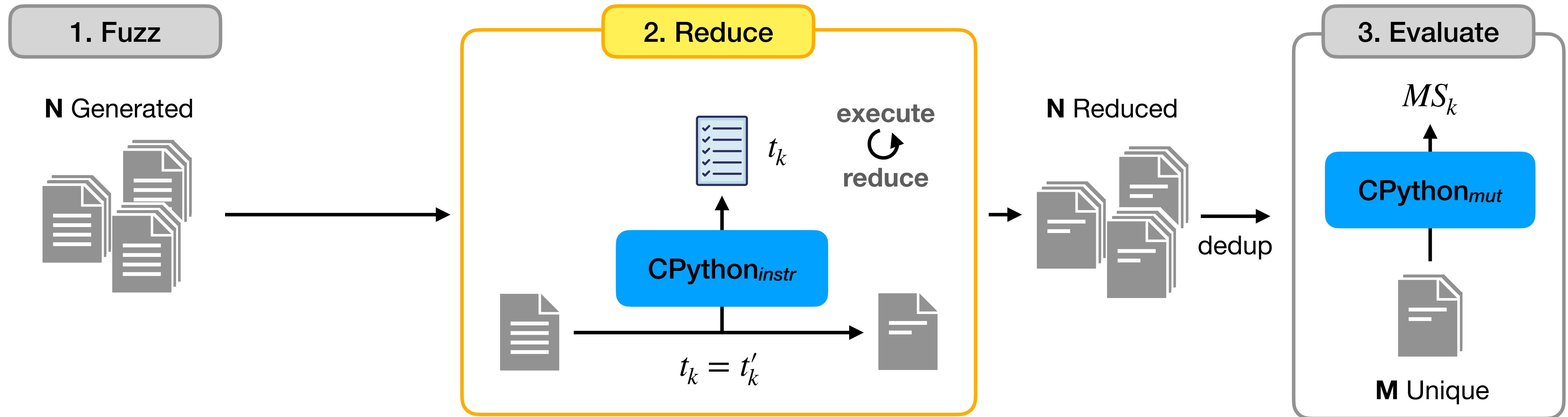


Methodology

Reduction

Perses_[2]: language agnostic program reducer

Uses grammar to maintain well-formedness



[2] Chengnian Sun, Yuanbo Li, Qirun Zhang, Tianxiao Gu, and Zhendong Su. 2018. Perses: syntax-guided program reduction. In Proceedings of the 40th International Conference on Software Engineering (ICSE '18). Association for Computing Machinery, New York, NY, USA, 361–371.

Evaluation

Setup

- Evaluate the **Effectiveness** of extracted test suites with
 - $N = 2000$ / $K = 0, 1, 2, 4, 8, 16, \infty$
 - Diversity: Mutation Score
 - Minimality: Performance, Size

Evaluation

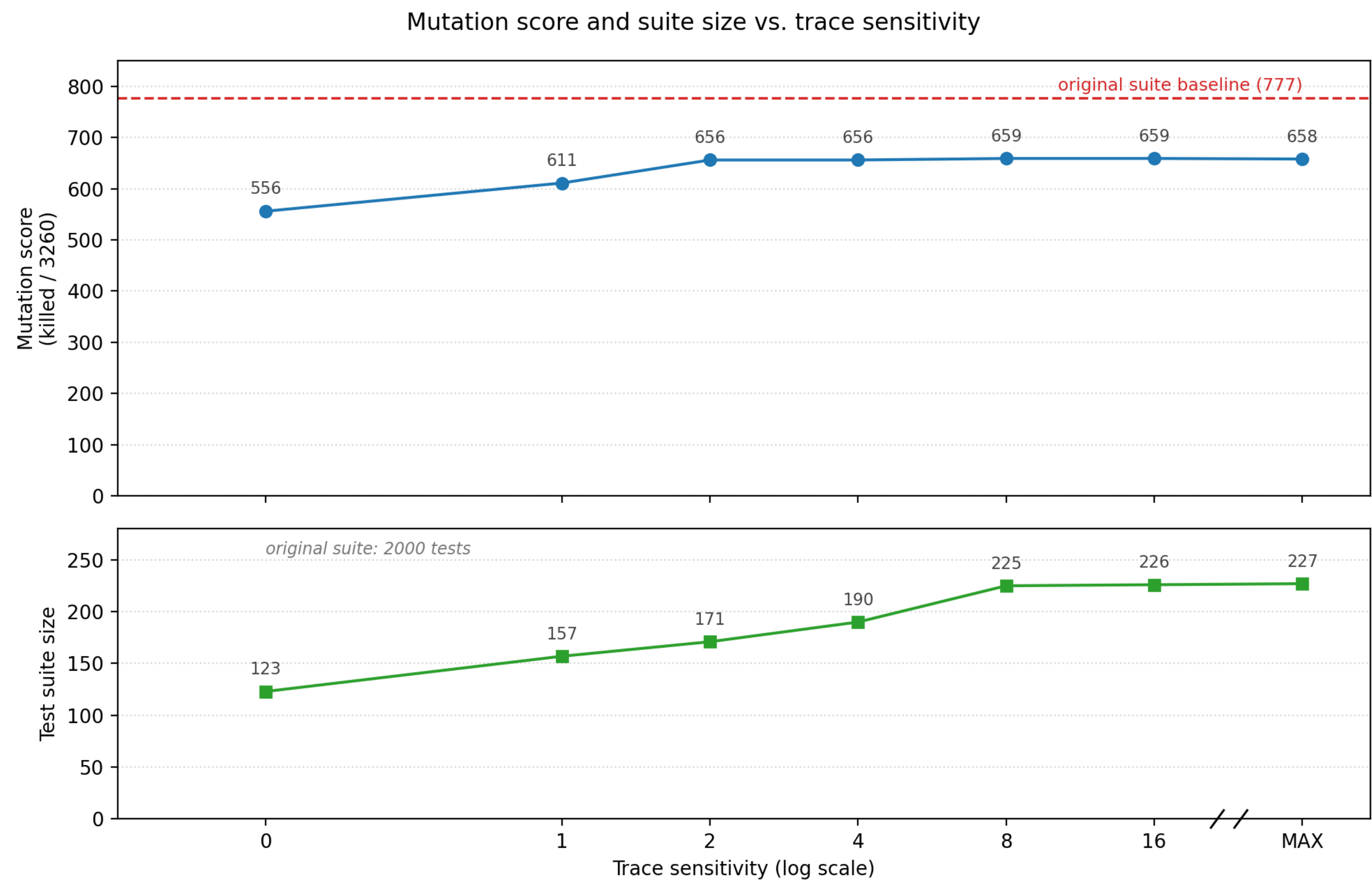
Diversity: Mutation Score

1. **Operators:** 10 representative operators following *Barbosa et al* [3]
 2. **Super-mutant:** Embed all into a single executable, and select at runtime
 3. **Targets:** Mutate functions executed at least once throughout all tests
 4. **Kill criteria:** Compare timeout, exit code, and exception type
- In total, we generated **3,260 relevant mutants** for evaluation.

[3] Barbosa, Ellen Francine, José Carlos Maldonado, and Auri Marcelo Rizzo Vincenzi. "Toward the determination of sufficient mutant operators for C." *Software Testing, Verification and Reliability* 11.2 (2001): 113-136.

Evaluation

Diversity: Mutation Score

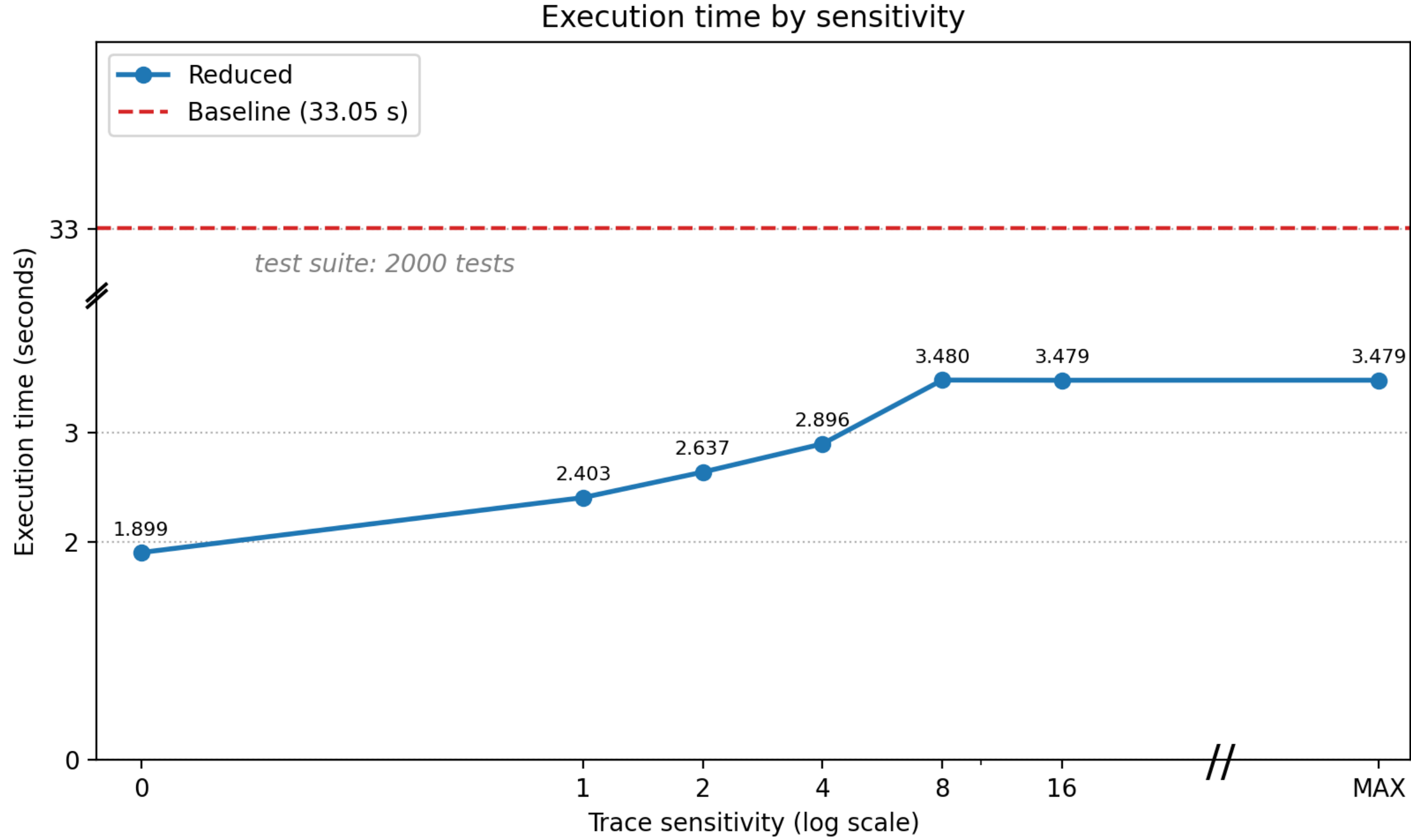


MS

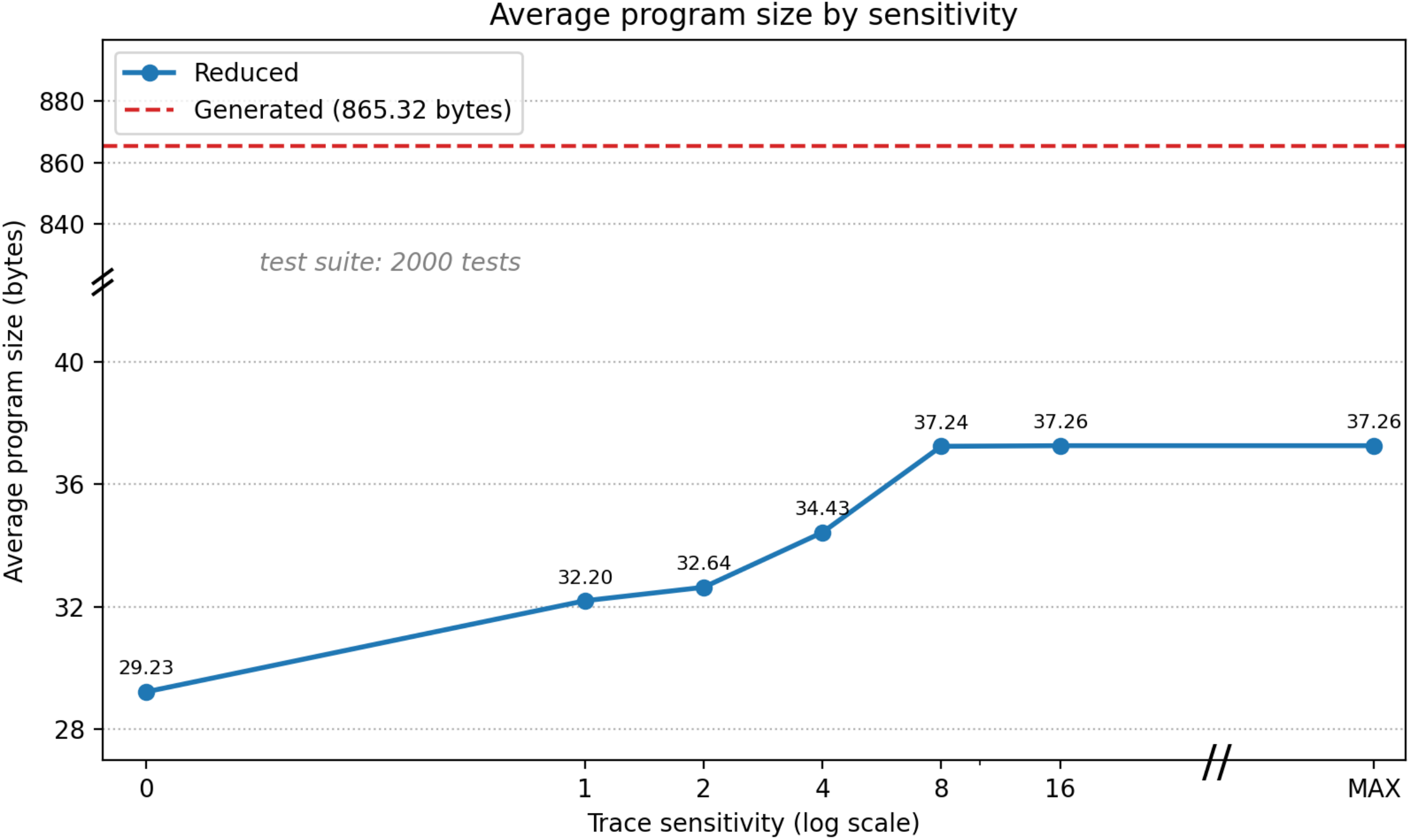
#. of tests

Evaluation

Minimality: Performance & Size



Execution Time



Avg. Program Size

Conclusion & Discussion

- + Execution-path-sensitive reduction preserves **diversity** within an **optimal** number of tests ($K=2$)
- Mutation Testing is ill-fitted for a negative test suite
- Lacks generality: Negative test suite, meta-programs
- **Threats to validity**: grammar rules for fuzzing, mutation target selection